



RAPPORT DE STAGE

MASTER RECHERCHE

SAGITTAIRE

Socio-communicative Abilities by Gesture Improvement Through Temporal
Active Interaction in Robotic Environments

Amélie LEGELEUX

15/01/18 - 30/06/18

Brest

Tuteurs :

Mihai POLCEANU

Cédric BUCHE

Pierre DE LOOR

Lab-STICC - IHSEV, ENIB



Remerciements

Je tiens à remercier le CERV pour m'avoir accueillie lors de mon stage ainsi que toutes les personnes ayant participé à son bon déroulement.

Je remercie tout particulièrement Monsieur BUCHE de m'avoir offert cette opportunité de stage et de m'avoir aidée durant toute sa durée. Merci également à Monsieur DELOOR pour ses conseils lors de l'avancement de mes travaux.

Je remercie également Monsieur POLCEANU, post-doc avec qui j'ai principalement travaillé, pour son partage de connaissances et son aide lors des diverses étapes du stage.

Finalement, je remercie les autres stagiaires Master au CERV pour leur soutien et leur bonne humeur.

Sommaire

<i>Introduction</i>	<i>1</i>
<i>Chapitre 1 : Étude bibliographique</i>	<i>2</i>
1.1 Perception de l'humain	2
1.2 Apprentissage et prise de décision des comportements	5
1.3 Génération d'actions en temps réel	7
1.4 Discussion	8
<i>Chapitre 2 : Solution proposée</i>	<i>12</i>
2.1 Développement d'un robot d'accueil	12
2.1.1 Prise en main du robot Pepper	12
2.1.2 Dialogue avec Pepper	13
2.1.3 Appel téléphonique de Pepper	15
2.1.4 Amélioration de l'appel téléphonique	15
2.2 Navigation vers les bureaux	16
2.2.1 Cartographie du CERV	16
2.2.2 Applications "Explore/Places/Patrol"	17
2.2.3 Algorithme de Dijkstra	18
2.2.4 Navigation dans le CERV	18
2.2.5 Nouvelle navigation	19
2.3 Étude de la base de données	20
2.3.1 Base de données UE-HRI	20
2.3.2 Récupération de la pose 2D en Python	21
2.3.3 Récupération de la pose 2D en C++	22
2.3.4 Traitement des données	23
2.4 Modèle d'apprentissage des mouvements	25
<i>Chapitre 3 : Résultats et discussion</i>	<i>26</i>
3.1 Démonstration du robot d'accueil	26
3.2 Évaluation et résultats du réseau de neurones	27
3.3 Travaux futurs	29
<i>Conclusion</i>	<i>30</i>
<i>Annexes</i>	
<i>Bibliographie</i>	

Table des figures

1.1	Estimation de la position et l'orientation du visage (en bleu) et direction du regard (en vert) avec OpenFace [Baltrušaitis et al., 2016]	3
1.2	Estimation de poses en 3D [Martinez et al., 2017]; première colonne : image 2D en entrée; deuxième colonne : réalité en 3D; troisième colonne : prédiction en 3D	3
1.3	Le modèle VNect [Mehta et al., 2017]	4
1.4	Le principe du modèle [Cao et al., 2017]	5
1.5	Architecture du modèle TDBN [Sukhbaatar et al., 2011]	6
1.6	Architecture d'un LSTM [Olah, 2015]	7
1.7	Modèle [Oh et al., 2015]	7
1.8	Le principe du modèle DyadGan [Huang and Khan, 2017]	8
2.1	Robot humanoïde Pepper [Aldebaran, 2018]	12
2.2	Affichage sur la tablette de Pepper lors de la reconnaissance du nom d'une personne du CERV	14
2.3	Pepper lors d'un appel téléphonique	15
2.4	Cartographie du CERV	16
2.5	Graphe créé sur la carte du CERV	17
2.6	Pseudo-code de l'algorithme de Dijkstra	18
2.7	Position et portée des lasers de Pepper [Aldebaran, 2018]	19
2.8	Position des capteurs infrarouges de Pepper [Aldebaran, 2018]	20
2.9	Pose 2D obtenue avec le modèle en Python	21
2.10	Joints du robot Pepper	23
2.11	Ordre des joints de la pose 2D	23
3.1	Pepper, robot d'accueil au CERV	27
3.2	Résultat d'une configuration du modèle; optimisateur : Adam, taux d'apprentissage : 0.1, taille de la couche cachée : 18, taille d'une séquence : 1000, taille d'un batch : 64	27
3.3	Valeurs de sortie et d'objectif du joint 0 lors de l'apprentissage et de la validation	28
3.4	Affichage pour la comparaison des tests du modèle	28

Liste des tableaux

1.1	Comparaison des modèles	9
2.1	Comparaison des vitesses de calcul des 2 versions du modèle de pose 2D	22

Introduction

Les robots se développent de plus en plus dans divers domaines tels la médecine, l'Armée et l'éducation. On retrouve également des robots dans les foyers afin de remplacer les aspirateurs ou les tondeuses. Les robots sociaux sont plus difficiles à implémenter du fait de l'interaction avec des humains. Avec l'avancement de l'intelligence artificielle, les robots acquièrent de nouvelles capacités telles la reconnaissance d'objets ou la compréhension de la parole.

Les interactions entre humains sont trop complexes pour être généralisées en catégories. Dans une interaction, la parole, l'intonation, les gestes non-verbaux et les émotions sont de nombreux paramètres qui influent sur son déroulement. Les robots doivent apprendre tous ces aspects afin d'avoir une interaction robot-humain la plus proche possible d'une interaction humain-humain. Pour cela, le robot peut apprendre en regardant des interactions entre humains et reproduire ce qu'il a vu. Cette technique s'appelle l'apprentissage par imitation. Cependant, il est compliqué de reproduire fidèlement les mouvements d'un humain. Un robot et un humain n'ont pas le même corps et donc pas les mêmes capacités de mouvements.

Mon stage s'est déroulé au Centre Européen de Réalité Virtuelle (CERV), centre de recherche rattaché à l'ENIB¹. C'est au sein de l'équipe IHSEV, Interactions Humains-Systèmes et Environnements Virtuels, du Lab-STICC² que mon stage s'est effectué.

Afin de s'insérer dans les projets de l'équipe IHSEV, mon stage a pris comme nom, SAGITTAIRE : Socio-Communicative Abilities by Gesture Improvement Through Temporal Active Interaction in Robotic Environments. Le stage s'inscrit dans le projet Sombrero et la Robocup@home qui tendent à développer les capacités d'interaction entre humain et robot. L'objectif de mon stage était de générer des mouvements non-verbaux d'un robot. Ces mouvements sont au moins aussi importants que le langage pour créer une interaction crédible avec un humain. La prise en compte du langage ne s'est effectuée que lors du développement du robot d'accueil. Le robot était capable d'aider des visiteurs à trouver une personne dans le CERV. Par la suite, la création d'un modèle devait permettre de générer des mouvements non-verbaux en temps réel en fonction des mouvements de la personne en face de lui.

Pour mener à bien ces travaux, une étude bibliographique (Chapitre 1) a été nécessaire afin de déterminer les modèles existants qui pourraient être utilisés, et afin d'acquérir de nouvelles connaissances. Ensuite, le stage a démarré et des solutions (Chapitre 2) ont été trouvées pour répondre aux problématiques de génération de mouvements lors d'une interaction avec un humain. Enfin, l'évaluation des résultats et les perspectives (Chapitre 3) ont été établies.

1. École Nationale d'Ingénieurs de Brest

2. Laboratoire des Sciences et Techniques de l'Information, de la Communication et de la Connaissance

Chapitre 1

Étude bibliographique

L'étude bibliographique est une étape essentielle lors de la création d'un modèle scientifique. Elle montre l'étendu du domaine ainsi que les connaissances et limitations actuelles. Pour générer des mouvements en temps réel et en fonction du comportement de la personne en face, il est nécessaire d'utiliser certains modèles existants. Les modèles ont été divisés en 3 catégories : la perception, la décision et l'action. Ces trois étapes décrivent les phases du modèle de génération de mouvements.

1.1 Perception de l'humain

La technique habituelle pour apprendre des mouvements humains est d'annoter des vidéos. Les catégories de mouvements sont donc préalablement définies. Cependant, avec des grosses bases de données, il est impossible de labelliser toutes les données manuellement. Un autre désavantage est l'erreur humaine. Si les données d'entrée d'un réseau de neurones sont mal annotées, l'apprentissage sera mauvais. Le meilleur moyen d'avoir une bonne perception est d'apprendre des caractéristiques sur des vidéos sans l'aide de l'humain. Ce procédé s'appelle l'apprentissage non-supervisé. Malheureusement, cet apprentissage est très compliqué à mettre en place dans ce contexte donc la plupart des modèles utilisent un apprentissage supervisé ou semi-supervisé. L'apprentissage semi-supervisé consiste à annoter seulement une partie des données. Cet apprentissage est donc un bon compromis entre l'apprentissage non-supervisé et l'apprentissage supervisé.

OpenFace [Baltrušaitis et al., 2016] est un outil open source qui est capable de détecter la position et l'orientation d'un visage, d'estimer la direction du regard ainsi que des unités d'action en temps réel (Figure 1.1). Un CLNF¹ et 3 couches de CNN² permettent de détecter des visages. Le CLNF capture les variations des contours de visage. Pour se souvenir d'un visage dans le temps, un CNN a été ajouté. Les sorties sont les contours de visages, les paramètres de forme, les unités d'action détectées et les vecteurs pour le regard. OpenFace peut détecter plusieurs visages dans des images et vidéos. Cet outil a été construit de manière à ce qu'il soit facilement intégrable dans d'autres projets.

1. Conditional Local Neural Fields
2. Convolutional Neural Network

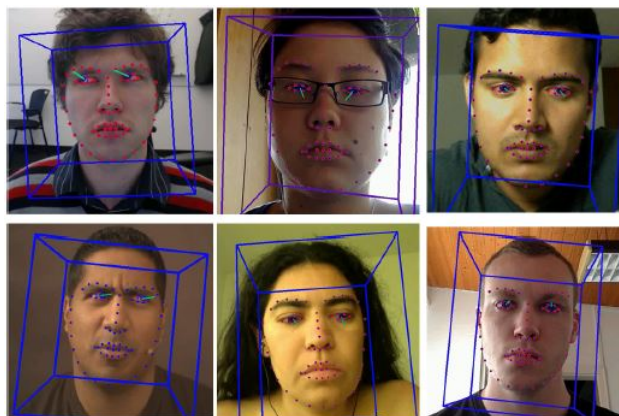


FIGURE 1.1 – Estimation de la position et l’orientation du visage (en bleu) et direction du regard (en vert) avec OpenFace [Baltrušaitis et al., 2016]

Une autre possibilité pour percevoir les mouvements est de représenter les joints en 3D [Martinez et al., 2017]. Le développement des algorithmes a rendu possible l’obtention d’une pose 3D à partir d’images 2D (Figure 1.2). Le modèle est simple avec seulement 2 CNN. La couche d’entrée augmente la dimension à 1024 et la couche de sortie produit une estimation de la pose en 3D. Les sorties de chaque CNN influencent le modèle en sortie, ce sont les connections résiduelles. Elles réduisent le temps d’apprentissage et les erreurs. Comme chaque image est traitée indépendamment, l’estimation est relativement fluide. Ce modèle est robuste au bruit. Le problème de cette architecture est qu’elle ne fonctionne pas lorsqu’une personne n’est pas visible en entier sur une image. Une autre limitation est la librairie de poses et le manque de vérification en sortie.

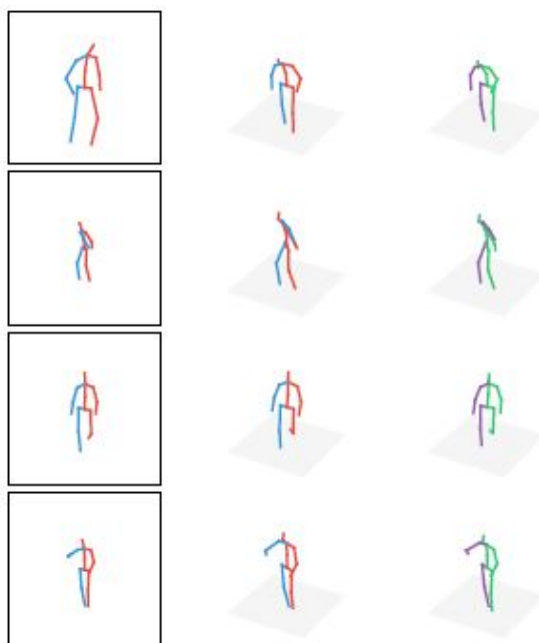


FIGURE 1.2 – Estimation de poses en 3D [Martinez et al., 2017] ; première colonne : image 2D en entrée ; deuxième colonne : réalité en 3D ; troisième colonne : prédiction en 3D

Le modèle VNect [Mehta et al., 2017] donne une estimation d’une pose 3D en temps réel à partir d’une simple caméra RGB (rouge, vert, bleu). Le modèle, illustré sur la figure 1.3, se décompose en 4 parties :

- une boîte de délimitation de suivi,
- une régression CNN,
- un filtre temporel,
- un raccord avec un squelette.

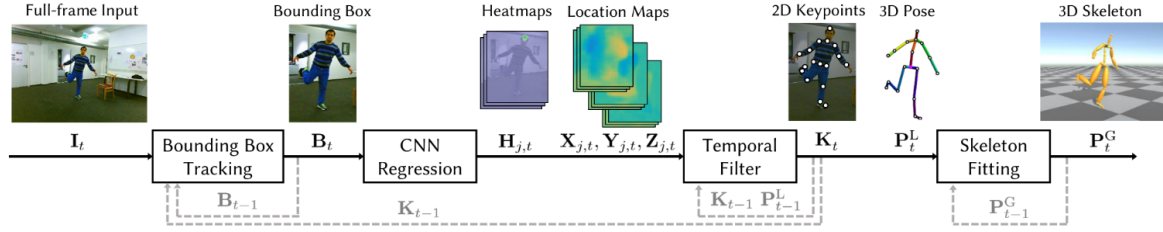


FIGURE 1.3 – Le modèle VNect [Mehta et al., 2017]

La boîte de délimitation de suivi diminue la taille de l'image afin de n'avoir plus qu'une image minimale contenant la personne. Cette délimitation suit la personne dans le temps. Cette partie est constituée de CNN. Le modèle fonctionne même sans avoir une délimitation stricte. La régression CNN extrait la position des joints en 2D et en 3D dans des cartes (carte d'énergie et carte de position) avec un réseau de neurones contenant 50 couches. Cette partie a été entraînée avec une base de données contenant des poses 3D de personnes annotées. Le filtre temporel combine les cartes obtenues précédemment afin d'obtenir les joints en 2D et en 3D. La dernière partie combine les joints en 2D/3D avec un squelette afin d'obtenir une estimation de la pose 3D. Cette estimation est temporellement stable et relative à la caméra donnée en entrée. Le modèle a donné des résultats comparables aux modèles existants et parfois meilleurs. Par rapport aux autres modèles qui prennent comme entrée une caméra RGB, le modèle [Mehta et al., 2017] a donné de meilleurs résultats. En comparaison avec les modèles ayant comme entrée une caméra RGB et un capteur de profondeur, le modèle montre des résultats similaires. Les résultats ont été meilleurs quand la personne se rapprochait d'objets de son environnement ainsi que pour des scènes avec la lumière du soleil et moins performant par rapport à l'estimation de profondeur. Le modèle est robuste face à des changements de scènes (scènes intérieures et extérieures), divers habits, diverses personnes, de nombreux types de caméra tel une simple caméra de smartphone et même avec des caméras de faible qualité. L'estimation de la pose 3D est obtenue en temps-réel (30Hz). Le modèle est limité à la détection d'une seule personne.

Percevoir des postures en 2D est beaucoup plus simple. [Cao et al., 2017] ont proposé un modèle capable de détecter des poses 2D de plusieurs personnes en temps réel. Leur architecture est composée de deux branches comme décrit sur la figure 1.4. La première branche, composée de CNN, détecte les joints des personnes. Le réseau crée un set de cartes de confiance de détection. Cela signifie qu'à chaque pixel est associé la probabilité qu'il s'agisse d'un joint. Les probabilités sont représentées par des couleurs sur l'image. Le rouge représente une forte probabilité et le bleu une faible probabilité. La seconde branche détermine les associations entre les joints. L'objectif est de relier les joints afin d'avoir des postures 3D. L'association de joints est faite à l'aide de PAF³. Il s'agit de champ qui représente la position et l'orientation de l'association en fonction de l'aspect visuel de l'image. Chaque pixel est donc traité indépendamment. Les deux branches sont séparées en phases. A la fin de chaque phase, les deux branches sont concaténées pour la prochaine phase. Cette division en phase permet de supprimer la confusion qu'il existe entre les

3. Part Affinity Fields

parties du corps qui sont symétriques. Ces deux branches sont associées à la fin du réseau afin de récupérer en sortie les joints en 2D de toutes les personnes de l'image d'entrée. L'évaluation des performances s'est effectuée avec un NVIDIA GeForce GTX-1080 GPU. Avec la base de données MPII⁴, l'architecture de [Cao et al., 2017] surpasse les autres méthodes de détection. Lors du « COCO⁵ Keypoints Channel », [Cao et al., 2017] ont eu les meilleurs résultats sauf sur la détection de personnes à très petite échelle. Le traitement d'une image avec 9 personnes dure 108.18ms. 8.8fps ont été atteint lors d'une détection de 19 personnes. Le modèle fonctionne également lorsque les personnes ne sont pas visibles en entier sur l'image.

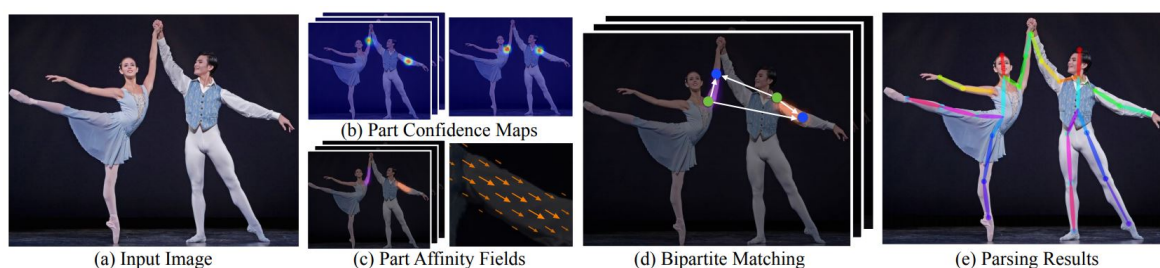


FIGURE 1.4 – Le principe du modèle [Cao et al., 2017]

1.2 Apprentissage et prise de décision des comportements

Le type de réseau de neurones pour l'apprentissage de comportements est choisi en fonction des données disponibles pour l'entraînement. Le réseau aura comme entrées des informations concernant la personne en face du robot et comme sorties les mouvements que le robot doit effectuer en temps réel.

Les réseaux de neurones ont besoin de beaucoup de boucles pour être entraînés. L'entraînement sur des vrais robots prend beaucoup de temps et peut causer des dommages au robot. Pour résoudre ce problème, [Hanna and Stone, 2017] ont proposé de simuler un robot physique pour entraîner un réseau avec des données physiques. Le modèle GAT (Grounded Action Transformation) a amélioré la vitesse de marche du robot NAO. Le GAT est composé de 2 réseaux de neurones ayant 3 couches chacun. Le premier modélise les dynamiques du robot et le second les nouvelles actions à effectuer. Le modèle a été implémenté sur les plate-formes SimSpark et Gazebo. Le problème de ce type de simulation est l'absence des forces externes réelles.

Le réseau TDBN (Temporal Deep Belief Network) a montré de robustes résultats dans la génération de mouvements [Sukhbaatar et al., 2011]. Un DBN⁶ est constitué de 2 couches de RBM⁷. La première partie du réseau contient des RBM pour chaque caractéristique et la seconde couche a un unique RBM pour une représentation compact. Un DBN est un bon modèle pour la

4. Max Plank Institut Informatik

5. Common Objects in Context

6. Deep Belief Network

7. Restricted Boltzmann Machine

reconnaissance de digits écrits manuellement et pour la reconnaissance d'objets. Dans [Sukhbaatar et al., 2011], le TDBN est entraîné avec des joints humains et génère le même mouvement. Un TDBN est composé de multiple RBM et d'un CRBM⁸ (Figure 1.5). Chaque RBM de la couche de multiple RBM correspond à une partie du corps comme la jambe gauche ou la jambe droite. Un RBM peut encoder et décoder des représentations de mouvements. Les multiples RBM réduisent les paramètres et diminuent le temps d'apprentissage. Plutôt que d'entraîner un gros réseau de neurones avec de nombreux liens et neurones, chaque RBM divise l'entraînement. Cependant, chaque RBM doit être entraîné avec la partie correspondante dans chaque image. Un CRBM a plus de couches qu'un RBM afin d'avoir une mémoire des états passés. Dans la génération de mouvements, un CRBM se souvient de quelques images du mouvement courant. Cela rend possible la génération de mouvements en temps réel. Les prochaines images dépendent des 3 images précédentes. Les entrées du CRBM sont les représentations de chaque RBM de la première couche. L'inconvénient de cette architecture est la coordination du corps entier. En effet, chaque partie du corps est évaluée indépendamment des autres. La stabilité d'un CRBM ne signifie pas que la génération rendra le robot stable.

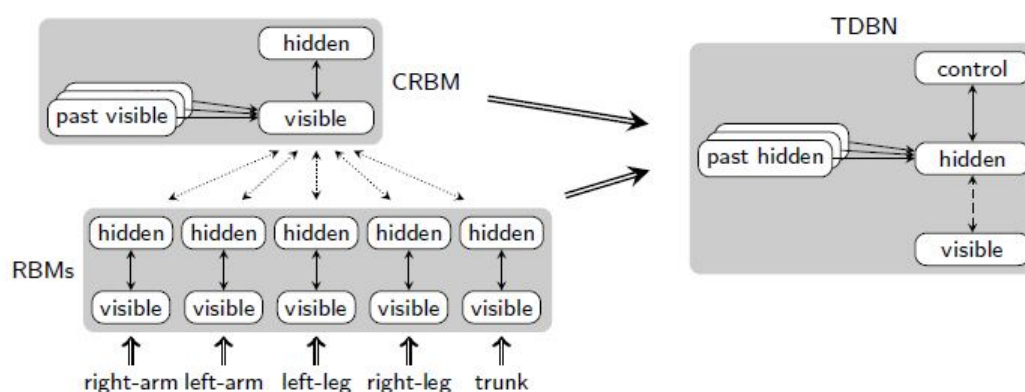


FIGURE 1.5 – Architecture du modèle TDBN [Sukhbaatar et al., 2011]

Le LSTM (Long Short-Term Memory) [Hochreiter and Schmidhuber, 1997] est un réseau de neurones de type RNN⁹. Ce type de réseau prend une séquence en entrée et génère une séquence en sortie. Pour chaque unité d'un LSTM (Figure 1.6), il y a 3 types de portail : un portail d'entrée (i_t), un portail de sortie (o_t) et un portail d'oubli (f_t). Le portail d'oubli détermine quelles données du réseau doit être oubliées. Chaque portail est un réseau de neurones multicouche. Un LSTM peut apprendre des séquences complexes telles la musique.

8. Conditional RBM

9. Recurrent Neural Network

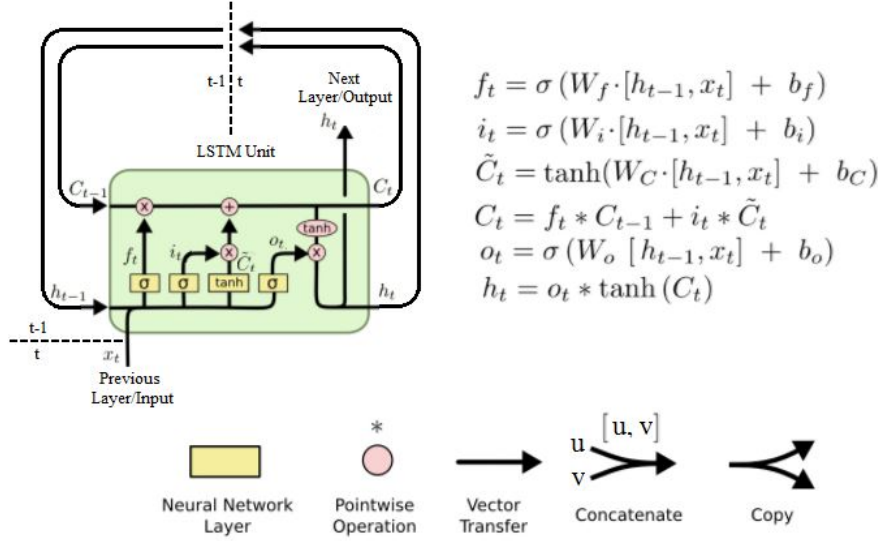


FIGURE 1.6 – Architecture d'un LSTM [Olah, 2015]

1.3 Génération d'actions en temps réel

Le fonctionnement en temps réel est parfois compliqué pour des réseaux de neurones. La prédiction du prochain mouvement en fonction du mouvement courant pourrait fluidifier les gestes. Un autre modèle fait de la génération d'expressions en fonction d'expressions de la personne en face.

Afin de modéliser un robot à travers le temps, [Oh et al., 2015] ont proposé 2 modèles : « FeedForward Encoding », « Recurrent Encoding ». Les modèles ont une partie encodage, une partie transformation et une partie décodage (Figure 1.7). La première architecture est composée de CNN pour la partie encodage et de CNN pour la partie décodage. La transformation est une simple formule mathématique. La seconde architecture dispose d'un LSTM pour la partie décodage. Les deux architectures ont donné de bons résultats pour une prédiction de 100 images. Les mauvaises prédictions surviennent quand un nouvel objet apparaît car il est difficile de prédire des événements inattendus. Le second problème est la détection de petit objet durant la phase d'apprentissage.

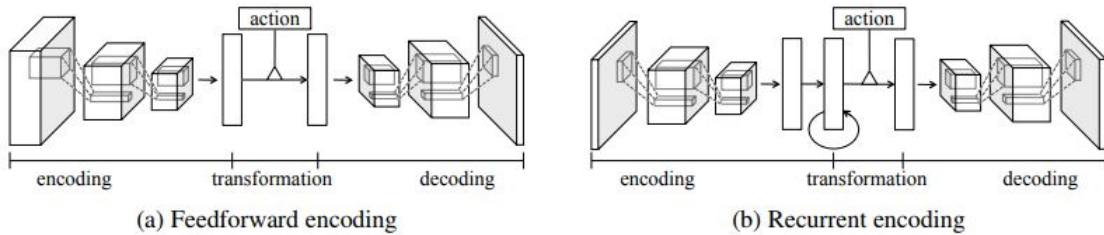


FIGURE 1.7 – Modèle [Oh et al., 2015]

Un GAN¹⁰ a été utilisé pour la génération d'expressions faciales dans une interaction dyadique. Dyadique signifie que l'interaction se déroule entre deux personnes. L'expérimentation consistait à interviewer des candidats à l'université afin d'évaluer leur niveau d'anglais. La génération concernait l'expression faciale dans des images. Le modèle proposé, DyadGAN [Huang and Khan, 2017] (Figure 1.8), est composé de 2 DC-GAN¹¹. Un DC-GAN [Radford et al., 2015] est un C-GAN¹² profond capable de créer une représentation des images avec un apprentissage non-supervisé et de générer des exemples d'images. Un C-GAN, GAN conditionnel, a la même structure qu'un GAN avec une entrée supplémentaire. Le premier DC-GAN a un vecteur de caractéristiques d'expressions comme entrée. Ce vecteur est calculé à partir des images d'entrée. Ce premier réseau génère des croquis. Le second génère des images à partir des sorties du premier réseau. L'évaluation est basée sur la distance Euclidienne entre les images réelles et les images générées. La génération d'expressions faciales dépend des expressions de la personne interviewée. Cette architecture ne génère pas des images en temps réel mais avec un RNN, les images générées pourraient se faire en continu.

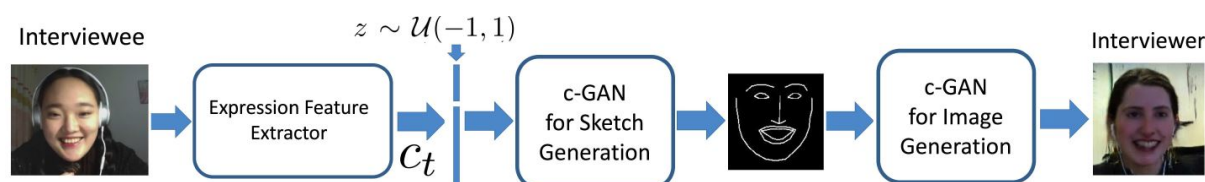


FIGURE 1.8 – Le principe du modèle DyadGan [Huang and Khan, 2017]

1.4 Discussion

Tous les travaux relatifs à mon stage sont listés dans le tableau 1.1. La première colonne donne la référence du modèle. Les trois autres colonnes représentent les 3 principales caractéristiques d'un agent : la perception, la décision et l'action. Pour chaque modèle est indiqué si l'apprentissage est supervisé, semi-supervisé ou non-supervisé. La dernière colonne mentionne si le modèle fonctionne en temps réel. Les modèles en gris sont ceux choisis lors de l'étude bibliographique et du stage mais non utilisés. Les modèles en vert sont ceux qui ont été utilisés pour réaliser la solution proposée.

Pour simplifier le modèle, les travaux ont été divisés en 3 catégories : perception de l'humain, apprentissage et décision des mouvements et la génération de mouvements. En réalité, c'est beaucoup plus complexe. Certaines approches rentrent dans plusieurs des catégories avec un seul réseau.

10. Generative Adversarial Network

11. Deep Conditional GAN

12. Conditional GAN

Perception			
Modèle	Description	Appren- tissage	Temps réel
[Mihoub et al., 2016]	Catégorisation de gestes Reconnaissance vocale	S N	Oui
[Huang and Mutlu, 2014]	Catégorisation de gestes Reconnaissance vocale Direction du regard	S S S	Oui
[Yang et al., 2015]	Extraction des objets et des catégories de mouvements	S	
[Baltrušaitis et al., 2016]	Extraction d'informations d'un visage	SS	Oui
[Martinez et al., 2017]	Estimation de poses 3D	SS	Oui
[Mehta et al., 2017]	Estimation de poses 3D	S	Oui
[Cao et al., 2017]	Estimation de multiple poses 2D	SS	Oui

Décision			
Modèle	Description	Appren- tissage	Temps réel
[Mihoub et al., 2016]	DBN : classification d'unités d'interaction	S	Oui
[Huang and Mutlu, 2014]	Réseau bayésien dynamique	S	Oui
[Hanna and Stone, 2017]	Simulation pour entraîner des réseaux de neurones	S	Oui
[Sukhbaatar et al., 2011]	Génération robuste de mouvements	S	Oui
[Lasson et al., 2017]	Classification de mouvements pour la gé- nération de mouvements	S	Oui
[Hochreiter and Schmidhu- ber, 1997]	LSTM	SS	Oui

Action			
Modèle	Description	Appren- tissage	Temps réel
[Mihoub et al., 2016]	Arbre de jonction : regard et gestes	S	Oui
[Huang and Mutlu, 2014]	Arbre de jonction : regard et gestes	S	Oui
[Yang et al., 2015]	Arbre d'analyse	S	
[Alletto and Rigazio, 2017]	Génération d'une carte d'énergie au niveau du pixel	N	Oui
[Jason et al., 2016]	Prédiction du mouvements fluides	N	Oui
[Oh et al., 2015]	Prédiction de mouvements	N	Oui
[Kober and Peters, 2010]	Apprentissage par renforcement	SS	Oui
[Wan et al., 2017]	Estimation de la pose 3D d'une main	SS	Oui
[Huang and Khan, 2017]	Génération d'images d'expression faciale en réponse à une expression humaine	SS	Non

S : supervisé, SS : semi-supervisé, N : non-supervisé

TABLE 1.1 – Comparaison des modèles

Pour percevoir les mouvements de l'humain, [Martinez et al., 2017] détecte très bien les mouvements en 3D. La dimension 3D est très utile pour pouvoir créer le modèle entièrement. Avec une vidéo contenant 2 personnes (ou 2 vidéos contenant chacun une personne), il suffit d'indiquer quelle personne le robot doit imiter et quelle personne sera la personne que le robot verra. Avec ce modèle, on peut extraire les postures 3D et entraîner un réseau de neurones avec des données afin d'imiter au mieux les mouvements humains dans une interaction. Cependant, le modèle ne fonctionne qu'avec une seule personne dans une vidéo et seulement si la personne est visible en entier. De plus, le modèle nécessite l'utilisation d'un autre modèle qui extrait une pose 2D. [Martinez et al., 2017] ajoute une dimension au résultat d'un premier modèle. Dans les faits, l'utilisation de ces 2 modèles a été trop complexe pour être testée.

Le modèle VNect [Mehta et al., 2017] est très intéressant du fait qu'il extrait en temps réel une pose 3D à partir d'une simple caméra. Ce modèle fonctionne si la base de données contient des images de personnes que le robot doit imiter, ce qui n'est pas le cas. De plus, le modèle nécessite l'utilisation du logiciel Matlab qui n'est pas disponible au laboratoire. Ce modèle n'a donc pas pu être testé mais pourrait être très intéressant dans le cas où la base de données disponible conviendrait.

OpenFace [Baltrušaitis et al., 2016] est un outil créé pour être facilement réutilisable et a donné de bons résultats en temps réel. Ce modèle n'a pas été testé car il extrait seulement des informations des visages mais pourrait remplacer ou compléter le modèle [Cao et al., 2017].

[Cao et al., 2017] extrait en temps réel plusieurs poses 2D à partir d'une vidéo, et même avec des personnes non visibles en entier. Ce modèle est donc plus intéressant à tester car plus complet. Cependant, il ne peut pas être utilisé dans le but d'extraire des joints qui devront être générés par le robot car la pose est en 2D. Avec la base de données utilisée lors du stage, ce modèle convenait parfaitement. Un répertoire Github permet de tester le modèle et de l'utiliser dans d'autres projets. Le modèle est disponible dans plusieurs langages : Python, Matlab et C++. Le modèle en Python a été testé en premier puis le modèle en C++. Le modèle qui fonctionne le mieux en temps réel est le modèle codé en C++.

Le modèle de [Sukhbaatar et al., 2011] est très utile lorsque les données d'entrée sont similaires aux données de sortie du réseau. La base de données utilisée ne permet pas d'avoir des entrées du même type que les sorties. Dans le cas d'un apprentissage avec simplement des vidéos, le modèle pourrait être intéressant.

Afin de faire le lien entre les données d'entrée et de sortie de la base de données, le réseau de neurones utilisé est un LSTM [Hochreiter and Schmidhuber, 1997]. Ce type de réseau fonctionne avec des séquences comme données d'entrée et de sortie, ce qui est le cas de la base de données disponible.

Avec la base de données, le modèle de [Hanna and Stone, 2017] n'est pas utile. En effet, les mouvements générés proviennent directement de mouvements réalisés par le robot. Dans le cas

où le réseau est entraîné avec seulement des vidéos en entrée, ce modèle sera intéressant pour vérifier si l'entraînement du réseau de neurones donne des mouvements réalisables par un robot.

Pour générer des mouvements en temps réel, l'idéal est de prédire les mouvements afin d'avoir des gestes plus fluides. Pour la perception, le modèle utilisé fonctionne en temps réel. Un LSTM fonctionne également en temps réel. Prédire des mouvements pourraient améliorer leur fluidité. La prédiction ne peut se faire que si un modèle est déjà établi. [Oh et al., 2015] permettrait de faire cette prédiction.

Le modèle de [Huang and Khan, 2017] est capable de générer des images d'expressions faciales selon l'expression humaine courante. Ce modèle n'utilise pas de robot mais pourrait être testé afin de vérifier s'il peut être adapté à la problématique du stage.

Pour la majorité des modèles choisis, l'apprentissage est non-supervisé ou semi-supervisé. Ce critère est essentiel car les données ne peuvent pas toutes être labellisées. Le fonctionnement en temps-réel est également important car le robot doit réagir immédiatement au comportement de l'humain en face de lui. L'objectif du stage est donc d'assembler et d'ajuster ces modèles afin d'améliorer les compétences sociales du robot Pepper.

Chapitre 2

Solution proposée

L'objectif du stage est de développer les capacités sociales d'un robot. Pour cela, le robot aura comme fonction première l'accueil de visiteurs au CERV. Après avoir pris en main la plateforme de programmation de Pepper, le robot a été capable de discuter, de téléphoner et de naviguer dans le CERV pour amener un ou des visiteur(s) au bureau demandé. De base, le robot a des mouvements pré-programmés qui ne s'adaptent pas à l'interaction en court. L'interaction n'est donc pas toujours compréhensible et réaliste. La création d'un modèle à base de réseau de neurones devrait donner au robot l'aptitude à réaliser des mouvements non-verbaux lors d'interactions avec des humains. Avec de nombreux exemples d'interactions, le robot devrait être capable d'apprendre afin d'avoir le comportement adapté à la personne en face de lui. Ce modèle, ajouté au robot d'accueil réalisé précédemment, rendra le robot plus crédible lors d'une interaction.

2.1 Développement d'un robot d'accueil

2.1.1 Prise en main du robot Pepper

Le robot utilisé lors du stage est le robot Pepper, développé par SoftBank Robotics, Aldebaran (Figure 2.1). Pepper est un robot humanoïde conçu pour guider, accueillir, renseigner et discuter avec des personnes.

Avec ses 1.21 mètres et ses 29 kg, Pepper est un concentré de technologies :

- Carte mère : processeur Atom E3845, CPU Quad core, fréquence d'horloge de 1.91Hz et RAM 4GB DDR3
- Connectivité : Wifi et prise Ethernet RJ45
- Tablette : 1280*800 pixels
- 2 haut-parleurs, 4 microphones
- 2 caméras 2D : 640x480 pixels à 30fps ou 2560x1920 à 1fps



FIGURE 2.1 – Robot humanoïde Pepper [Aldebaran, 2018]

- 1 caméra 3D : 1280x720 à 15fps
- 1 caméra de profondeur : composée de 2 caméras 2D, 2560x720 à 15fps
- Leds : au niveau des yeux, des oreilles et des épaules
- 1 centrale inertielle
- 6 lasers, 2 capteurs infra-rouges, 2 sonars
- Moteurs : tête, bras, hanches, genoux, 3 roues
- 5 capteurs tactiles : 1 sur chaque main, 3 sur le dessus de la tête
- 3 pare-chocs/boutons : au niveau des 3 roues

Le but du robot d'accueil est d'accueillir des visiteurs, de discuter avec eux et de les guider vers les personnes qu'ils souhaitent rencontrer. Pour cela, Pepper doit être capable d'écouter et de répondre, de proposer d'appeler la personne demandée au téléphone ou d'amener le visiteur au bureau de la personne recherchée.

Avant de démarrer la programmation, le robot a dû effectuer une mise à jour (version Naoqi 2.5 [Aldebaran, 2018]). Le système d'exploitation de l'ordinateur portable utilisé lors du stage est Ubuntu 16.04. Ensuite, une connexion doit être effectuée entre le robot et un ordinateur. Pour ce faire, il existe 2 possibilités : une connexion filaire ou une connexion Wifi. Pour la connexion filaire, un câble Ethernet RJ45 relie un ordinateur et le robot. Après avoir configuré un routeur, une connexion Wifi a pu être établie.

Pour programmer Pepper, il y a le logiciel Choregraphe ou la possibilité d'écrire du code en Python ou en C++. Le logiciel Choregraphe a été très utile pour tester rapidement les possibilités du robot comme la parole, la reconnaissance de mot, les mouvements et la visualisation des caméras. Ce logiciel offre également des outils de réglages tels le niveau de son, le niveau de batterie, l'activation et désactivation des moteurs. La programmation se fait grâce à des boîtes qu'il faut relier entre elles. Par la suite, du code en Python a permis de contrôler le robot. Le choix du langage était lié aux diverses librairies et modèles disponibles et utilisés par la suite, notamment pour les réseaux de neurones.

Pour le robot d'accueil, le programme était lancé directement dans le robot. Le modèle pour la génération de mouvement demandait trop de calculs pour être lancé dans le robot. Les calculs ont donc été faits depuis un ordinateur et le résultat (les mouvements à effectuer), seront envoyés sur le robot pour qu'il agisse en conséquence.

2.1.2 Dialogue avec Pepper

Pepper est doté d'un mode "Autonomous Life" que l'on peut activer ou désactiver. Ce mode donne des capacités au robot afin de le rendre vivant et réactif. Le robot a des mouvements de base même lorsqu'il est seul. Quand il entend ou voit une personne, il tourne la tête et si besoin son corps pour être face à la personne. Quand il a détecté un visage, les leds de ses yeux sont

éclairées en rose. Il réagit également à d'autres stimulus : la tablette, les capteurs sur la tête, les pare-chocs/boutons. Quand on parle au robot, il a des mouvements de tête et/ou de bras. De base, ce mode ne permet pas au robot de parler.

Le "Basic Channel" donne des capacités de discours au robot. Le robot peut répondre à des questions de base (voir Annexe A). Avant de parler au robot, il faut qu'il ait détecté un visage (leds des yeux roses) et qu'il soit en mode écoute (leds des yeux tournent en bleu). Quand Pepper parle, il a des mouvements non-verbaux qui rendent l'interaction plus réaliste.

Le vocabulaire auquel Pepper réagit est donc limité au "Basic channel". Pour étoffer son langage, il est possible d'ajouter du vocabulaire. Pepper a donc assimilé de nouveaux mots : les noms des différentes personnes du CERV, "CERV", "Ferme l'image", "Arrête le programme". Pour ajouter des réactions en fonction du nouveau vocabulaire, la classe *reactToSpeech* définit une callback liée à l'évènement "WordRecognized". Une callback est une fonction de rappel qui est donnée comme paramètre à une autre fonction. Quand un mot est reconnu, la callback est appelée. C'est ainsi que l'on peut associer des réactions à des mots ou groupes de mots.

Le premier test lors de la reconnaissance de mots a été de faire avancer le robot selon un certain mot. Ensuite, Pepper devait afficher une image en fonction d'un mot clé. Une fois ces deux tests validés, il restait à ajouter les photos des personnes en fonction de leurs noms. Deux icônes dans les coins supérieurs ont été ajoutés afin de pouvoir sélectionner l'appel téléphonique ou la navigation. Les fonctions associées aux deux icônes seront implémentées par la suite.

Pour afficher plus qu'une simple image sur la tablette de Pepper, il est nécessaire de créer une page html. Cette page web permet d'afficher une image en fonction d'un paramètre donnée avec une URL. Ce paramètre correspond au nom de la personne demandée. Ainsi l'affichage correspond à la personne demandée et ajoute deux icônes pour l'appel et la navigation (Figure 2.2).

Ainsi, Pepper est capable de reconnaître le nom des personnes du CERV, d'afficher la photo de la personne demandée et de proposer d'appeler la personne ou de naviguer vers son bureau. Si aucune photo n'existe pour la personne demandée, une image de base est affichée à la place de la photo.



FIGURE 2.2 – Affichage sur la tablette de Pepper lors de la reconnaissance du nom d'une personne du CERV

2.1.3 Appel téléphonique de Pepper

Twilio [Twilio, 2018] est une librairie qui permet de passer des appels téléphoniques. Après avoir gratuitement créer un compte et obtenu un numéro de téléphone, on peut appeler des numéros de téléphone qui ont été préalablement validés. Cette plate-forme offre une API¹ afin de facilement passer un coup de téléphone si une connexion Internet est disponible.

Twilio offre la possibilité de rédiger le message retransmis au téléphone. Le discours suivant est donc transmis : "Bonjour, je suis Pepper. Des visiteurs vous attendent à l'accueil.". Si personne n'a décroché au bout de 30 secondes ou que la personne n'a pas écouté le message en entier, on considère que l'appel n'a pas abouti.

Après avoir tester Twilio sur un ordinateur portable, il faut tester sur Pepper. Suite à des soucis de proxy, Pepper a pu être connecté à Internet via un partage de connexion d'un téléphone portable. Pepper a donc été capable de passer un appel téléphonique au numéro correspondant à la personne demandée. Pour faire le lien entre l'icône de la tablette (page html) et l'appel téléphonique (code Python), des événements ont été créés. Ainsi, lorsque l'on touche l'icône "téléphone", un événement "Make a call" est déclenché, ce qui démarre l'appel téléphonique du code Python.

Pepper est maintenant capable de reconnaître le nom des personnes du CERV, d'afficher leur photo et de passer un appel téléphonique si le visiteur le souhaite.

2.1.4 Amélioration de l'appel téléphonique

L'appel téléphonique fonctionne mais n'est pas assez crédible sur un robot. Pour rendre l'interaction plus réaliste, le robot doit bouger, sonner, parler et réagir en fonction de l'appel téléphonique.

Le logiciel Choregraphe permet de bouger manuellement le robot et d'obtenir les valeurs courantes des moteurs. A l'aide de 9 postures, le mouvement du bras droit de Pepper imite un humain au téléphone. Ces valeurs sont ajoutées au code Python et donne un mouvement fluide sur le robot.

La sonnerie téléphonique est un élément essentiel lors d'un appel. Le son d'une sonnerie est donc émise par le robot lors du mouvement du bras jusqu'à la fin de l'appel téléphonique.



FIGURE 2.3 – Pepper lors d'un appel téléphonique

1. Application Programming Interface : ensemble de services (classes, méthodes, fonctions) offert par un logiciel

Quand l'appel est terminé, Pepper répète le message envoyé lors de l'appel puis descend le bras. Ensuite, selon si la personne a décroché ou non, le robot prévient le visiteur que la personne va bientôt venir ou qu'il/elle n'a pas répondu.

Pour rendre encore plus réaliste et compréhensible l'interaction, une image d'un téléphone est affichée lors de l'appel pour montrer que le robot est en train d'appeler et qu'il ne peut rien faire d'autre tant que l'appel n'est pas terminé, comme le montre la figure 2.3. De base, il y a une animation sur la tablette de Pepper. Au bout de 15 secondes, si le visiteur n'a pas cliqué sur un des deux icônes, l'image de la personne demandée disparaît. Afin de rendre encore plus utile le robot à l'accueil, un visiteur peut demander le plan du CERV et une image du plan s'affiche sur la tablette de Pepper.

2.2 Navigation vers les bureaux

2.2.1 Cartographie du CERV

Pour pouvoir naviguer dans le CERV, il faut avoir une carte. L'API Naoqi de Pepper dispose d'une fonction qui fait de l'exploration. A partir des divers capteurs du robot, Pepper explore son environnement dans un rayon donnée et en extrait une carte. Afin d'obtenir une carte la plus exacte possible, il ne faut aucun obstacle et que les portes des différents bureaux soient toutes ouvertes pour pouvoir les visualiser sur la carte.

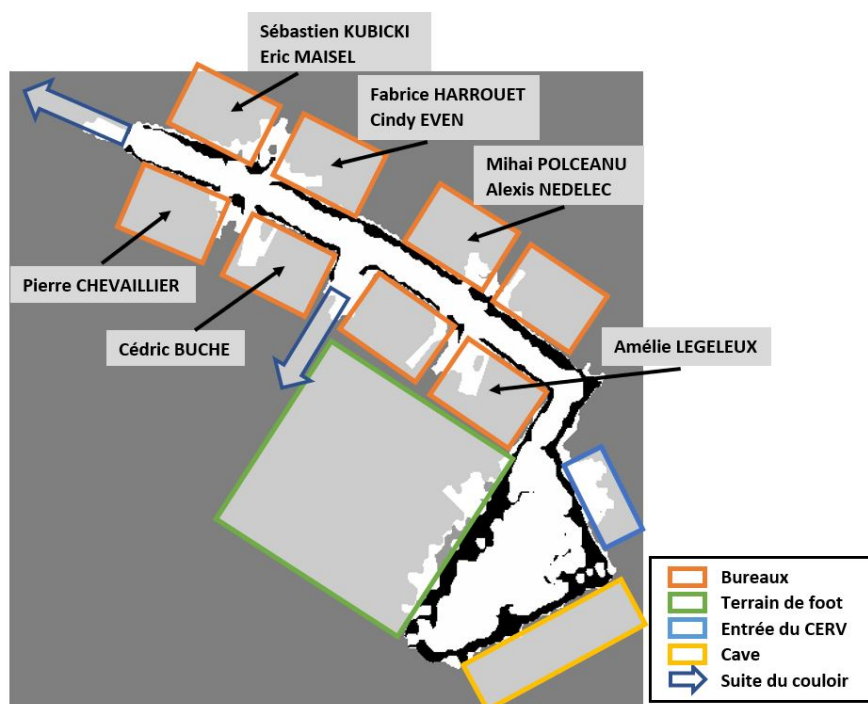


FIGURE 2.4 – Cartographie du CERV

La carte de la figure 2.4 est celle obtenue lors de l’exploration du CERV par Pepper. Les obstacles sont en noir et l’espace libre est en blanc. Le fond de l’image est en gris. Grâce à l’ouverture des portes, on peut visualiser correctement les différentes pièces. Le nom des personnes des différents bureaux sera utilisé par la suite pour donner des consignes au robot.

2.2.2 Applications "Explore/Places/Patrol"

Le répertoire Github *naoqi_navigation_samples* [Souchet, 2017] est composé de 3 applications :

- *Explore* permet de scanner une pièce et de créer une carte,
- *Places* charge la carte sélectionnée sur la tablette et crée des emplacements,
- *Patrol* active une patrouille dans une carte.

L’application *Explore* n’est pas utile car la carte du CERV est déjà créée. Afin d’avoir des emplacements dans la carte, l’application *Places* a été très efficace. Après avoir chargé la carte du CERV, il suffit de toucher des emplacements sur la tablette et leur ajouter un nom. Tous les lieux sont sauvegardés dans un fichier.

L’application *Patrol* a été utilisé lors de démonstrations. Une fois la carte chargée, on sélectionne successivement des emplacements en prononçant leur nom ou en touchant un lieu sur la carte. L’ordre de choix des emplacements correspond à l’ordre dans lequel le robot ira aux différents emplacements.

Intuitivement, la navigation a d’abord été testée avec la fonction *navigateToInMap*, utilisée par l’application *Patrol*. Cependant, cette navigation n’était pas satisfaisante. L’entrée dans le couloir du CERV était un problème. De part la protection contre les collisions, le robot était coincé au début du couloir, ne parvenant pas à y rentrer. Cette fonction était actuellement en cours d’implémentation chez SoftBank Robotics Aldebaran, donc la navigation devait être faite autrement. Pour faire naviguer le robot dans le CERV, il est donc indispensable de tout implémenter.

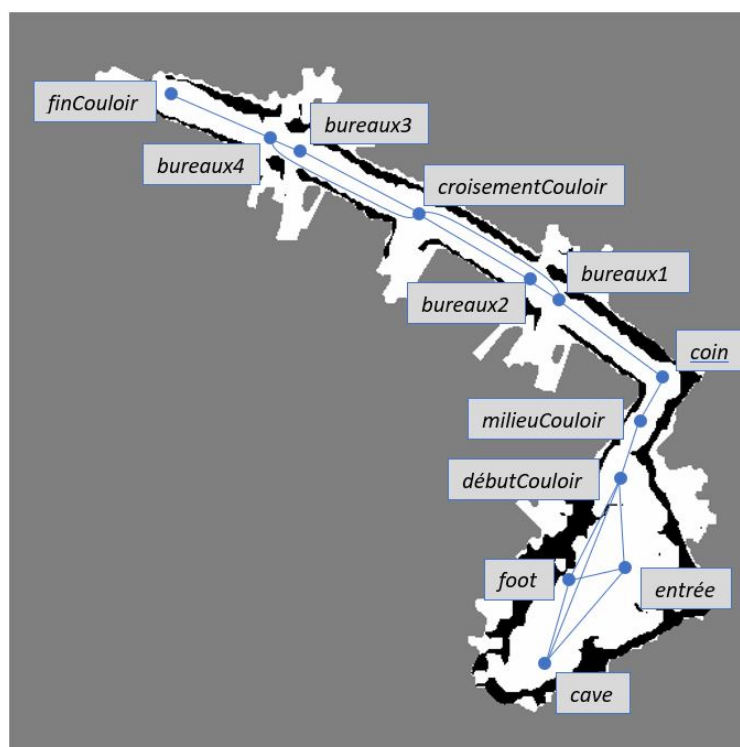


FIGURE 2.5 – Graphe créé sur la carte du CERV

Les lieux récupérés avec l'application *Places* crée un maillage sur la carte (Figure 2.5). Ces différents points, nommés et reliés ensemble, forment un graphe. Un algorithme de pathfinding, comme l'algorithme de Dijkstra, détermine le chemin de le plus court entre deux points.

2.2.3 Algorithme de Dijkstra

Le pathfinding a été réalisé avec l'algorithme de Dijkstra. Cet algorithme a été choisi car il est simple et donne de bons résultats. A chaque lieu de la carte du CERV est associé ses voisins et la distance qui les sépare. Le pseudo-code de la figure 2.6 suit l'algorithme de Dijkstra et renvoie le chemin le plus court entre les points *depart* et *arrivee* donnés en paramètre de la fonction.

```
fonction Dijkstra(graphe,depart,arrivee) :  
  noeuds_visites  $\leftarrow$  0  
  pour chaque nœud du graphe :  
    distance_min[noeud]  $\leftarrow$  infini  
    predecesseur[noeud]  $\leftarrow$  vide  
  fin pour  
  distance_min[depart]  $\leftarrow$  0  
  tant que nombre noeuds_visites < nombre de nœuds dans graphe  
    noeud  $\leftarrow$  nœud non visité de distance la plus faible  
    pour chaque successeur du nœud :  
      si nœud n'est pas dans noeuds_visites et  
        distance_min[successeur] - distance_min[noeud] > distance nœud-successeur :  
          distance_min[successeur]  $\leftarrow$  distance_min[noeud] + distance nœud-successeur  
          predecesseur[successeur]  $\leftarrow$  nœud  
      fin si  
    pour  
      noeuds_visites  $\leftarrow$  ajouter nœud  
    fin tant que  
    chemin  $\leftarrow$  arrivee  
    tant que depart n'est pas dans chemin :  
      chemin  $\leftarrow$  ajouter en première place predecesseur[chemin[0]]  
    fin tant que  
  retour chemin
```

FIGURE 2.6 – Pseudo-code de l'algorithme de Dijkstra

2.2.4 Navigation dans le CERV

Grâce au pathfinding, on obtient le chemin le plus court pour aller à un point donné. Pour le mouvement, les points sur la carte du CERV ont été choisis de manière à ce que le robot n'ait pas besoin de faire de courbes.

Avant de faire naviguer le robot, il faut qu'il se localise. Ensuite, le robot récupère régulièrement sa position dans la carte et détermine le point le plus proche. L'algorithme de Dijkstra détermine le chemin le plus court entre le point courant et l'objectif. Le robot suivra donc le chemin obtenu. Un point est caractérisé par un rayon qui détermine la distance à laquelle le robot estime qu'il a atteint ce point. En fonction de cette distance, le robot va essayer d'atteindre son point courant ou non.

Pour atteindre un point, le robot tourne sur lui-même puis se déplace en ligne droite. En fonction de l'orientation actuelle du robot et de l'orientation qu'il doit atteindre, l'angle est calculé et donné comme consigne. Après la rotation, la distance qui le sépare du point est calculée et donnée comme consigne pour effectuer un mouvement en avant. Une boucle sur le chemin à parcourir permet au robot d'atteindre les différents points avant d'arriver à son objectif.

2.2.5 Nouvelle navigation

La navigation réalisée pose des problèmes dans le couloir et surtout dans l'angle du couloir. Lorsque le robot se localise dans la carte, si la localisation est décalée par rapport à la réalité, les commandes de mouvement seront décalées. Le robot peut donc rentrer dans un mur. La navigation ne permet donc pas au robot de naviguer dans le CERV. Il est donc indispensable de refaire la navigation et l'évitement d'obstacles afin de faire déplacer le robot dans le couloir.

Dans un premier temps, l'interface ROS a permis de vérifier le fonctionnement des capteurs et caméras de Pepper en temps réel. Les caméras fonctionnent très bien. Les sonars détectent parfois des obstacles quand il n'y en a pas. Les lasers détectent très bien les obstacles.

Pour l'ancienne navigation, faire tourner le robot et aller en ligne droite demandaient 2 lignes de commande de consigne. Chaque 4ms, une nouvelle instruction est donnée au robot. Cette instruction comprend la rotation et le mouvement en avant. Le robot tourne donc en même temps qu'il avance. Sauf dans le cas où la destination se trouve derrière lui, il tournera avant d'avancer. Le reste de la navigation est identique.

Les distances de sécurité bloquaient le robot si un obstacle se trouvait trop près et le robot mettait un certain temps avant de sortir de ce mode. Pour remédier à ce problème, Mihai Polceanu m'a aidé à implémenter un nouvel évitement d'obstacles. La sécurité de base du robot a été désactivée et complètement repensée. Les divers lasers couvrent 3 zones de 60 degrés chacun (Figure 2.7). Plus un obstacle est proche, plus le robot reculera. En moyennant toutes les distances à parcourir en fonction des lasers, une valeur en x et une en y

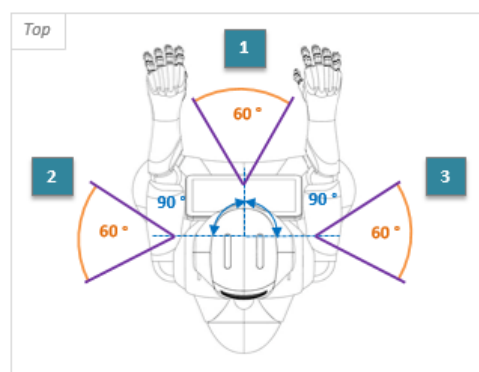


FIGURE 2.7 – Position et portée des lasers de Pepper [Aldebaran, 2018]

sont obtenues et ajoutées au mouvement de base du robot lors de la navigation. Ainsi pendant la navigation, si Pepper se retrouve trop près d'un mur, il va reculer et l'éviter. Les 3 zones que couvrent les lasers n'étaient pas suffisantes pour empêcher le robot d'aller dans l'angle du couloir. Les capteurs infrarouges (Figure 2.8) ont donc été utilisés pour couvrir certaines zones d'ombre que les lasers ne peuvent couvrir. Le robot peut alors se déplacer dans le couloir sans toucher les murs.

Cette nouvelle navigation s'adapte donc en fonction de l'environnement. Pepper connaît le chemin qu'il doit parcourir et évitera les obstacles sur son chemin. Si une personne se trouve sur le chemin du robot et lui bloque le chemin, Pepper s'arrêtera et attendra que la voie soit libre avant de reprendre son mouvement en avant.

Cette navigation a été associée à l'icône correspondant sur la tablette. Pour rendre plus réaliste l'aide apportée par le robot, de nouvelles caractéristiques ont été ajoutées lorsque le robot arrive au bureau demandé. Un fois le robot arrivé au bureau, il fait un demi-tour sur lui-même afin d'être face au(x) visiteur(s). Ensuite, à l'aide d'un mouvement de la main, il indique l'emplacement du bureau. Si le bureau demandé se trouve sur sa gauche, le mouvement se fera avec sa main gauche et inversement. A ce mouvement est associé un texte oral. Après cela, le robot attend 3 secondes puis va revenir tout seul à l'accueil du CERV.

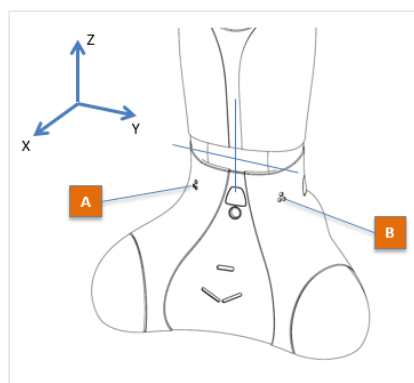


FIGURE 2.8 – Position des capteurs infrarouges de Pepper [Aldebaran, 2018]

2.3 Étude de la base de données

2.3.1 Base de données UE-HRI

La base de donnée du projet Sombrero ne contenait pas assez de données pour entraîner un réseau de neurones. De plus, les données ne contenaient pas toujours des vidéos de l'intervieweur et l'interviewé, ce qui est indispensable pour l'apprentissage du réseau.

La base de données [Ben-Youssef et al., 2017] contient des données qui ont été récoltées lors d'interviews entre Pepper et des visiteurs. Le scénario était le suivant :

- Phase d'acceptation : le visiteur doit renseigner son adresse mail, confirmer qu'il est majeur, accepter que des données soient récoltées ainsi que d'autres accords optionnels ;
- Phase de bienvenue : Pepper se présente et donne des instructions.
- Phase de dialogue : Pepper pose des questions comme se présenter, leur livre préféré ;
- Phase concombre : Pepper présente sa vision de la technologie en annonçant par exemple

que pour lui la différence entre un concombre et un humain est le visage ;

- Phase d'enquête : 15 questions à propos de l'interaction avec le robot.

Le robot était placé dans un lieu public où les participants étaient libres d'interagir lorsqu'ils le souhaitaient et de partir. Aucune personne n'était informée de la présence du robot au préalable afin que l'interaction enregistrée soit la plus spontanée possible.

Lors de ces interactions, de nombreuses données ont été recueillies. Au niveau le plus bas se trouve les données de base du robot : vidéos (deux en 2D et une en 3D), audio, sonars, lasers. A un niveau un peu plus haut, il y a d'autres informations comme celles sur le visage (localisation des yeux, nez, bouche, rotation de la tête, direction du regard, degré d'ouverture des yeux), la distance et la position du visiteur par rapport au robot, les différentes valeurs des joints du robot. Au niveau le plus haut sont regroupées des données plus sophistiquées telles les expressions faciales, le dialogue du visiteur, l'estimation de l'âge.

Au total, 124 interactions ont été recueillies. 54 d'entre-elles sont disponibles pour des travaux de recherche. Une interaction se commence lorsque le robot détecte un mouvement et se termine quand le robot ne détecte plus personne. Une interaction peut contenir des hommes et/ou des femmes et une ou plusieurs personnes. Une interaction dure entre 4 et 15 minutes. Chaque interaction, contenant toutes les données correspondantes, est sauvegardée dans un *rosbag*. Les 54 *rosbags* forment une archive de plus de 400Go.

2.3.2 Récupération de la pose 2D en Python

Avant de tester le modèle [Cao et al., 2017], de nombreuses librairies et packages ont dû être installés. Ce travail a pris beaucoup de temps car il y a eu des problèmes d'installation, de chemin et de paramètres de compilation.

Comme tout le code précédent et la librairie pour faire un réseau de neurones sont en Python, le modèle a d'abord été testé dans sa version Python. Le modèle fonctionne avec une seule image donnée en entrée. Le premier test s'est donc fait avec les images de base fournies avec le code Python (Figure 2.9) et ensuite avec une nouvelle image récupérée sur Internet. Afin de faire tourner le modèle sur une vidéo, une boucle appelle le modèle plusieurs fois et l'affichage est actualisé pour obtenir une

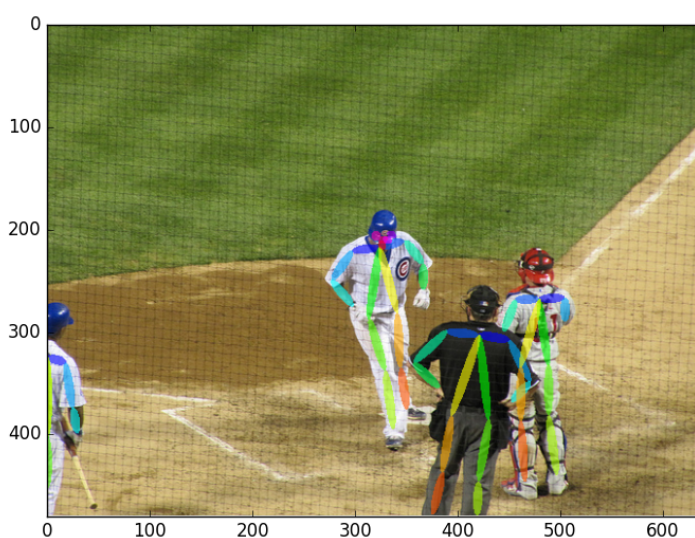


FIGURE 2.9 – Pose 2D obtenue avec le modèle en Python

estimation des poses 2D en temps réel. Le test avec une vidéo a été effectuée avec la vidéo de la webcam de l'ordinateur. Le test suivant utilisait la vidéo de la caméra de Pepper. Ensuite, le modèle a été testé sur mon ordinateur personnel, ayant une meilleure carte graphique. L'ordinateur du stage a un NVIDIA Quadro 1000M GPU et mon ordinateur contient un NVIDIA GeForce GTX-1060 GPU. Avec mon ordinateur, le temps de calcul a été divisé par 2.

2.3.3 Récupération de la pose 2D en C++

Le modèle codé en C++, OpenPose, est la version qui fonctionne le mieux en temps réel. Mais comme le modèle sera appelé depuis un code Python, il a fallu utiliser un *wrapper* qui fait le lien entre du code Python et du code C++. Dans le code C++ du modèle, du code C a fait le lien entre les fonctions qui seront appelées dans le code Python et celle du code C++. La compilation du code C++ crée une librairie qui pourra être importée dans le code Python. La librairie *ctype* permet de charger une librairie pour l'utiliser ensuite. Cela offre la possibilité d'utiliser les fonctions écrites précédemment en C. Le modèle OpenPose importe de nombreuses librairies pour fonctionner, ce qui est à prendre en compte lorsque l'on crée un *wrapper*. Pour faciliter le travail lié aux diverses librairies, PyOpenPose [Paschalis and Lanius, 2018], un *wrapper* pour le modèle OpenPose, a été utilisé.

Le tableau 2.1 regroupe les résultats du modèle en Python et OpenPose obtenus avec différentes vidéos en entrée et sur 2 ordinateurs différents. Les résultats sont significatifs. OpenPose fonctionne beaucoup plus rapidement que le modèle codé en Python. Avec une meilleure carte graphique, la vitesse de traitement de calculs est diminuée. De plus, plus l'image est grande, plus le temps de calcul augmente. Par la suite, la vidéo récupérée par la caméra de Pepper sera de taille 480x640 pixels, cellules vertes dans le tableau. Dans un premier temps, le modèle sera testé sur l'ordinateur du stage. Si le temps le permet, le modèle sera également testé sur mon ordinateur personnel afin d'accélérer le temps de calcul.

Modèle en Python			
Vidéo en entrée	Taille de l'image (pixel)	Ordinateur du stage	Ordinateur personnel
Caméra de Pepper	480x640	0.13fps	0.26fps

OpenPose (Modèle en C++)			
Vidéo en entrée	Taille de l'image (pixel)	Ordinateur du stage	Ordinateur personnel
Vidéo du modèle	1280x720	1.6fps	3.8fps
Caméra de Pepper	240x320	2.3fps	5.2fps
Caméra de Pepper	480x640	2.2fps	5.0fps

TABLE 2.1 – Comparaison des vitesses de calcul des 2 versions du modèle de pose 2D

Sur mon ordinateur personnel, une vidéo de 15 secondes met 41.40 secondes à être traitée par le modèle. Il est possible de ne pas traiter toutes images afin de fonctionner en temps réel. Avec cette option du modèle, l'ordinateur a mis 15.18 secondes pour traiter la vidéo. Cette option ne peut pas fonctionner avec un réseau de neurones car les images doivent être périodiques pour que le système fonctionne. Cette option pourra être testée dans un second temps afin de comparer et vérifier si cela ne diminue pas les performances. Les tests avec la caméra de Pepper ont été réalisés en Wifi. En Wifi, la caméra peut au maximum transmettre les données à 11fps pour des images de 480x640 pixels. En Ethernet, la vitesse est de 30fps. Comme Pepper doit bouger, le modèle ne peut fonctionner qu'avec du Wifi.

OpenPose possède une option de visualisation 3D de la pose. Pour que cette option fonctionne, il faut 2 caméras pour avoir une pose en 3D. Cette option fonctionne avec un certain type de caméra. Pour tester avec d'autres caméras, il est plus compliqué de tester la visualisation 3D. La visualisation 3D serait très utile si la base de données contient une vidéo avec une personne que Pepper doit apprendre à imiter.

2.3.4 Traitement des données

Pour entraîner un réseau de neurones, il faut les bonnes données d'entrée et de sortie. Le réseau sera entraîné avec comme données d'entrée les angles des joints des poses 2D extraites d'images vues par Pepper et les joints du robot. Les joints peuvent être simplement extraits de la base de données. Les poses 2D doivent être calculées pour chaque image.

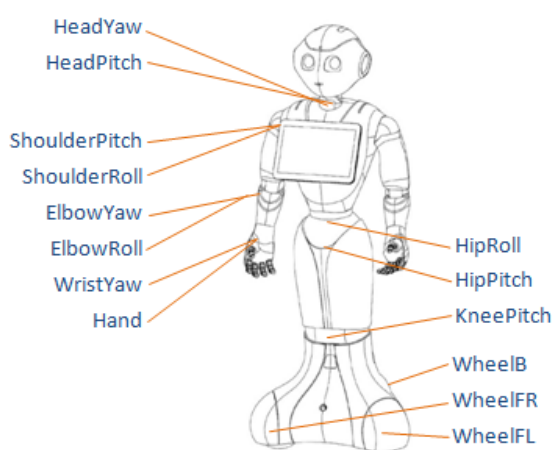


FIGURE 2.10 – Joints du robot Pepper

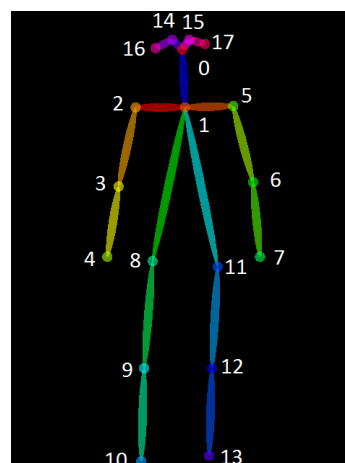


FIGURE 2.11 – Ordre des joints de la pose 2D

Il y a 17 joints sur le robot (Figure 2.10) dans l'ordre suivant : lacet de la tête, tangage de la tête, tangage de l'épaule gauche, roulis de l'épaule gauche, lacet du coude gauche, roulis du coude gauche, lacet du poignet gauche, main gauche, roulis des hanches, tangage des hanches, tangage des genoux, tangage de l'épaule droite, roulis de l'épaule droite, lacet du coude droit,

roulis du coude droit, lacet du poignet droit, main droite, roue devant à gauche, roue devant à droite, roue derrière. La pose 2D extrait la position de 18 joints selon l'ordre des numéros de la figure 2.11.

A chaque image récupérée dans la base de données est extrait un vecteur contenant un vecteur pour chaque personne détectée. Le vecteur d'une personne contient la position x , y et le degré de confiance pour chaque joint. Afin d'avoir des données de taille identique, la personne la plus grosse sur l'image, c'est-à-dire la personne la plus proche de la caméra, sera la seule dont on récupérera les joints. Pour chaque image, le vecteur de pose 2D aura comme taille 36 (position x et y pour les 18 joints). Si une personne n'est pas visible en entier sur l'image, les joints non visibles prennent la valeur 0. Si aucune personne n'est visible sur l'image, le vecteur ne contiendra que des zéros.

Afin de faciliter le traitement des données, le vecteur de pose 2D et le vecteur des joints du robot seront de type *numpy array*. Les données de la base de données ne sont pas périodiques. Il faut donc définir une période pour avoir des données régulièrement. En moyenne sur un *rosbag*, la période minimum entre 2 données de joints est de 72ms et de 120ms entre 2 images de la même caméra. Le pas de temps défini est de 60ms. Si aucune nouvelle donnée n'est disponible dans les 60 nouvelles millisecondes, la dernière donnée est dupliquée.

Pour trier toutes les données, chaque *rosbag* a été traité indépendamment puis le vecteur de joints et le vecteur de pose 2D de chaque 60ms ont été stockés dans un fichier Pickle. A chaque *rosbag* est donc associé un fichier Pickle. Un fichier Pickle contient un vecteur de joints de taille (nombre de données, 17) et un vecteur de poses 2D de taille (nombre de données, 36). Les *rosbags*, c'est à dire les 400Go, ont été traités. L'espace libre de l'ordinateur du stage n'étant pas assez grande, le traitement des données a du être réalisé en 3 fois. Au total, il aura fallu 39 heures et 31 minutes pour créer les 54 fichiers Pickle, d'une taille totale de 440Mo.

Pour entraîner correctement le réseau, les angles de chaque joint des poses 2d ont été calculés. L'origine correspond au nez. Si le nez n'est pas détecté, c'est-à-dire des coordonnées x et y nulles, aucun autre angle n'est calculé. En effet, lors d'une interaction, le robot doit voir la personne et donc son nez avant d'interagir. En fonction des liens des joints de la pose 2d, les angles sont calculés. Ainsi, l'angle est relatif au joint précédent. Par exemple, pour un simple mouvement d'une épaule, la valeur de l'angle du coude ne sera pas modifiée.

Les angles et les joints du robot ont été normalisés afin d'être tous compris entre 0 et 1. Le réseau de neurones sera donc entraîné avec les angles des joints des poses 2d en entrée et les joints du robot correspondant comme sortie.

2.4 Modèle d'apprentissage des mouvements

Pour apprendre les mouvements, le réseau de neurones utilisé est un LSTM. Ce type de réseau permet d'apprendre des séquences, ce qui est très utile surtout dans un système temps réel. Le réseau prédira le prochain mouvement en fonction des mouvements précédents. La librairie utilisée pour créer le réseau est *PyTorch* [PyTorch, 2018].

Le réseau doit être entraîné avec les angles de la pose 2D en entrée et les joints du robot comme sortie. Les données doivent être séparées en 3 groupes :

- la base d'entraînement : pour entraîner le réseau ;
- la base de validation : pour ajuster les paramètres du réseau et pour vérifier si le modèle a été bien entraîné ;
- la base de test : pour valider ou non le modèle.

Les *rosbags* ont été répartis respectivement dans les bases : 50, 2 et 2. Afin d'accélérer les calculs, les données calculées et réparties dans les 3 groupes ont été exportées dans 3 fichiers Pickle différents. Ainsi, la récupération des données au début du programme ne prend que 13 à 15 secondes.

Pour entraîner le réseau, il faut définir la taille d'un batch et la taille d'une séquence. Une séquence est définie par une suite de données dans le temps. Dans ce cas, une séquence aura une taille de 1000. Comme les données ont été récupérées toutes les 60ms, cela représente une durée de 60s, soit une minute. Le batch a une taille de 64 (2^6). C'est-à-dire qu'il est composé de 64 séquences. Les séquences sont obtenues aléatoirement. Le fichier Pickle est choisi aléatoirement puis dans ce fichier est tiré au sort une séquence de 60s.

Les différents paramètres du réseau, appelés hyperparamètres, vont être ajustés afin d'améliorer le modèle. Les paramètres qui sont modifiables sont les suivants : la taille d'une séquence, la taille d'un batch, le nombre de couches cachées, le nombre de neurones dans les couches cachées et le taux d'apprentissage. Le réseau est actuellement en cours de test.

Chapitre 3

Résultats et discussion

Afin de vérifier si les objectifs du stage ont été atteints, les diverses étapes du stage ont été évaluées. Pour la partie concernant le développement du robot d'accueil, le programme créé correspond aux exigences du stage. Le réseau de neurones n'est pas terminé. Les résultats présentés correspondent donc aux avancées actuelles. La poursuite des travaux sera présentée dans un dernier temps.

3.1 Démonstration du robot d'accueil

Des démonstrations ont eu lieu à différentes phases du stage. Les démonstrations ont impliqué principalement des lycéens, des étudiants, des entreprises, des professeurs. Les capacités de Pepper se sont développées au fur et à mesure.

L'Humanoïd Open Brest est un événement sur la robotique qui s'est déroulé sur Brest. Des équipes françaises et allemande se sont affrontées lors de tournois de foot avec des robots autonomes. Pepper était également présent pour représenter la ligue *Robocup@home*. L'événement était ouvert au public et se tenait sur 3 jours. Des journalistes sont venus lors de l'Humanoïd Open Brest. Un reportage télévisé lors du journal régional ainsi que des articles de journaux ont décrit cet événement (voir Annexe B).

La version finale du robot d'accueil fonctionne très bien (Figure 3.1). Le robot a son vocabulaire de base et peut discuter avec des visiteurs. Si on touche le capteur de la main droite du robot, les réactions du vocabulaire qui ont été ajoutées s'activent. Quand on touche de nouveau le capteur, les réactions se désactivent. Le robot peut ainsi montrer le plan du CERV si le visiteur le souhaite. Lorsqu'un visiteur demande à Pepper la possibilité de voir une personne du CERV, Pepper montre la photo de la personne demandée. Ensuite, Pepper propose de l'appeler au téléphone ou de montrer le chemin jusqu'à son bureau. L'appel téléphonique ne fonctionne plus car la librairie Twilio était gratuite seulement lors d'une période d'essai. La navigation fonctionne très bien dans le couloir. Il est également possible de démarrer l'application *Patrol* en touchant le capteur de la main gauche de Pepper.



FIGURE 3.1 – Pepper, robot d'accueil au CERV

3.2 Évaluation et résultats du réseau de neurones

Le travail sur le réseau de neurones est toujours en cours. Les résultats suivants sont ceux obtenus lors de l'écriture de ce présent rapport. D'autres résultats seront obtenus avant la fin du stage.

Le modèle a été testé sur l'ordinateur du stage et sur mon ordinateur personnel. Il sera également testé sur d'autres ordinateurs ayant plus de mémoire avec la carte graphique. Les résultats ont été obtenus sur mon ordinateur sur CPU¹. Le modèle sur GPU² ne fonctionne pas encore.

La figure 3.2 représente les résultats d'une configuration du modèle : optimisateur Adam, taux d'apprentissage de 0.1, 1 couche cachée d'une taille 18, des séquence de 1000 et un batch de 64. L'abscisse correspond au nombre d'itération et l'ordonnée aux taux d'erreur. La courbe bleu correspond à l'entraînement et la courbe verte à la validation.

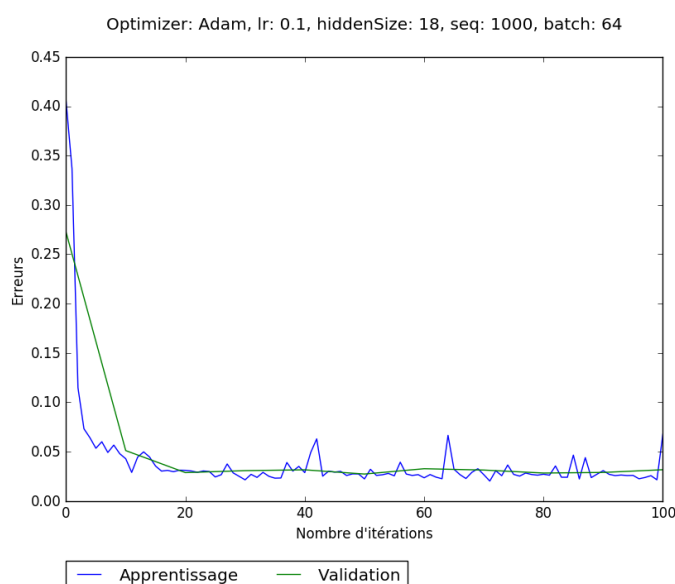


FIGURE 3.2 – Résultat d'une configuration du modèle ; optimisateur : Adam, taux d'apprentissage : 0.1, taille de la couche cachée : 18, taille d'une séquence : 1000, taille d'un batch : 64

1. Central Processing Unit
2. Graphics Processing Unit

La validation a été faite toutes les 10 itérations. Les courbes montrent une diminution des erreurs. La validation est très similaire à l'entraînement mais il faut vérifier à l'aide de courbes représentant la sortie et l'objectif, si le modèle a appris correctement.

Pour vérifier l'apprentissage, la figure 3.3 montre les valeurs du joint 0 après l'apprentissage sur la base d'entraînement et celle de validation. En vert sont représentées les valeurs réelles du joint 0. Les valeurs de sorties du réseau sont en bleu. Les valeurs ont été récupérées après toutes les itérations. Pour l'apprentissage, les valeurs de sortie devraient être quasi identiques aux valeurs d'objectifs. La validation devrait être légèrement moins bonne que pour l'apprentissage. On peut observer que sur les courbes, les valeurs de sortie ne correspondent pas du tout aux objectifs. Pour améliorer le réseau, il faut augmenter la taille de la couche cachée et entraîner le modèle sur un plus grand nombre d'itérations.

La figure 3.4 montre l'affichage prévu pour la comparaison des tests. Les courbes correspondent aux taux d'erreurs lors de l'apprentissage et lors de la validation. Pour l'instant, seul un test, et donc une seule courbe, est visible pour la comparaison. Dans le code est prévu l'affichage de plusieurs courbes correspondant à divers tests afin de comparer leur performance.

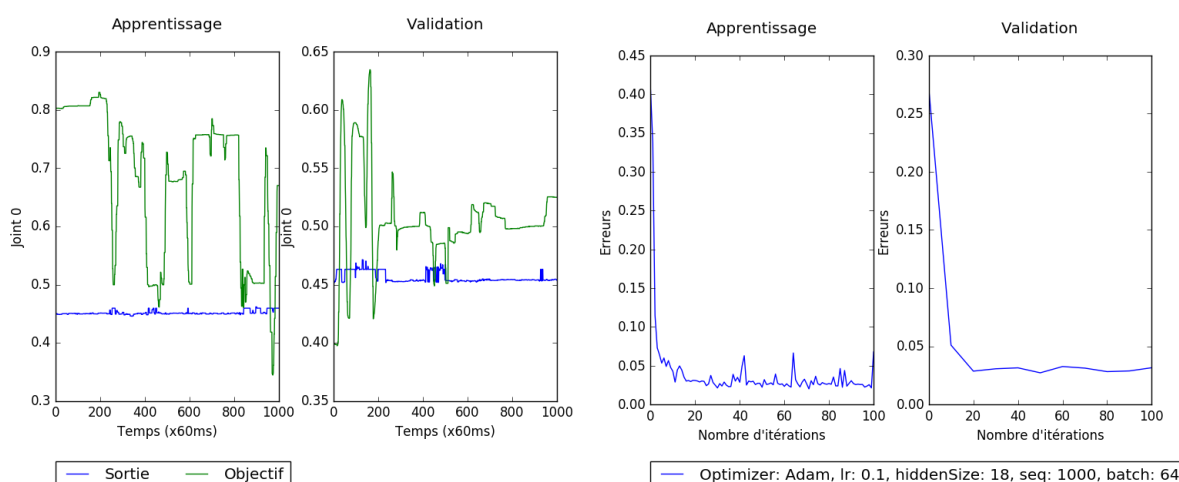


FIGURE 3.3 – Valeurs de sortie et d'objectif du joint 0 lors de l'apprentissage et de la validation

FIGURE 3.4 – Affichage pour la comparaison des tests du modèle

Le code correspondant au modèle a été écrit de manière à pouvoir être tester sur d'autres ordinateurs. Ainsi d'autres essais sur de nouveaux ordinateurs devraient permettre de résoudre les limitations actuelles. Pour l'instant, le modèle ne fonctionne que sur CPU et seulement avec un nombre faible de neurones et d'itérations. De nouveaux tests sont en cours pour améliorer le modèle.

3.3 Travaux futurs

Le réseau de neurones est toujours en construction. Les résultats de fin de stage pourront déterminer si l’objectif en terme d’apprentissage et de génération a été atteint. Le travail sur le robot d’accueil peut s’inscrire dans la Robocup@home. Au niveau du projet Sombrero, la fin du stage permettra de savoir si la contribution correspond aux objectifs du projet.

De nombreux autres modèles pourraient être testés voir ajoutés au modèle proposé afin d’améliorer les performances. Par exemple, le modèle OpenPose [Baltrušaitis et al., 2016] peut servir de nouvelle entrée ou être ajouté aux entrées afin d’affiner l’apprentissage. Le réseau n’apprendrait plus seulement à partir des joints mais également avec des informations du visage (orientation et position, unités d’action).

Avec une autre base de données contenant des vidéos de personnes à imiter et les vidéos de personnes à regarder, le modèle pourrait être entièrement repensé. Un modèle qui extrait une pose 3D donnerait une génération de mouvement fidèle à un mouvement humain. Le réseau de neurones pourrait être le modèle TDBN [Lasson et al., 2017] si la base de données contient des informations sur chaque joint autant pour l’entrée que pour la sortie du réseau.

Concernant le temps-réel, les modèles proposés par [Oh et al., 2015] pourraient permettre de générer des mouvements plus fluides. L’apprentissage du réseau pourrait utiliser [Hanna and Stone, 2017] afin de vérifier si les mouvements générés ne vont pas causer de dommage au robot.

Enfin, le modèle proposé par [Huang and Khan, 2017] ne s’applique pas au robot actuellement. Pourtant le principe de générer une image en fonction d’une autre pourrait peut-être être utilisé dans le cas de la génération de mouvements dans une interaction.

Conclusion

Ce stage a démarré par une étude bibliographique qui a permis de connaître les modèles existants dans le domaine et d'acquérir des connaissances sur des technologies pouvant servir ou utilisant de l'intelligence artificielle. Ensuite, l'implémentation d'un robot d'accueil m'a permis de prendre en main la plate-forme du robot Pepper. Le réseau de neurones a pour objectif d'améliorer les capacités d'interaction du robot en lui apprenant des gestes non-verbaux. Par rapport au planning prévisionnel, le déroulement du stage s'est déroulé de manière équivalente (voir Annexe C).

Des difficultés ont été rencontrées lors du stage comme lors d'une mise à jour du robot ou lors d'installations de bibliothèques. Tous ces obstacles ont pu être surmontés pendant le stage. Le robot d'accueil obtenu correspond parfaitement aux attentes. Concernant le modèle de génération de mouvement, les tests sont toujours en cours donc l'évaluation des résultats n'est pas terminée. Les capacités de Pepper en tant que robot d'accueil correspondent à des exigences de la Robocup@home. Les résultats concernant le projet Sombrero seront obtenus à la fin du stage.

De nombreuses activités parallèles ont jalonné le déroulement du stage. En plus des démonstrations de Pepper, j'ai également appris à faire des démonstrations des robots Nao sur le terrain de foot. L'Humanoïd Open Brest a été un événement marquant de mon stage autant au niveau des rencontres avec les autres équipes de foot qu'avec le public de Brest. La publication d'articles dans les journaux et sur d'autres plate-formes montrent le développement de la robotique en France. L'intelligence artificielle, thème du Master SIIA ³, connaît un essor depuis quelques années.

Ce stage m'a apporté de nombreuses connaissances sur les robots d'Aldebaran SoftBank Robotics, sur les réseaux de neurones ainsi que des capacités personnelles comme la capacité de parler en public lors des démonstrations. En parallèle des travaux du stage, des cours en ligne sur les réseaux de neurones m'ont apportée de plus solides connaissances sur leur fonctionnement. Les présentations de thèses et d'autres travaux dans le laboratoire fournissent un regard plus global sur les possibilités de la réalité virtuelle et de l'intelligence artificielle.

Actuellement, mon projet professionnel s'oriente vers un poste d'ingénieur au CERV. Le poste d'ingénieur devrait commencer en octobre et permettre de continuer de travailler sur les robots, autant Pepper que les robots Nao.

3. Systèmes Intelligents, Interactifs et Autonomes

Annexes

Liste des annexes

A Dialogue de base : "Basic Channel"

B Publications de l'Humanoïd Open Brest

C Diagrammes de Gantt prévisionnel et réel

ANNEXE A

Dialogue de base : "Basic Channel"

Pepper a un dialogue de base : le "Basic Channel". La liste de questions de base ci-dessous n'est pas exhaustive. Pepper peut reconnaître les questions sous une autre forme. Par exemple, Pepper peut comprendre "Comment puis-je t'appeler ?" ou "Comment tu t'appelles ?". La liste correspond à la version Naoqi 2.5.

Rencontre avec Pepper	A propos des applications
Comment ça va ?	Que sais-tu faire ?
Fais-moi coucou.	Quelles sont tes applications ?
Répétition	Niveau de batterie
Qu'est-ce que j'ai dit ?	Quelle est ton niveau de batterie ?
Tu peux répéter ?	Es-tu en charge ?
Connexion Internet	Date et temps
Quelle est ton adresse IP ?	Quelle heure est-il ?
Es-tu connecté à Internet ?	Quel jour on est ?
Langues	Volume
Quelle langue parles-tu ?	Parle moins fort.
Parle <nom d'une langue>.	Parle plus fort.
Est-ce que tu sais parler <nom d'une langue> ?	
Mouvements de Pepper	
Tends les bras (le bras gauche/droit).	Bouge la tête (à gauche/droite).
Lève les bras (le bras gauche/droit).	Tourne la tête (à gauche/droite).
Fais bouger tes mains (gauche/droite).	Lève la tête.
Peux-tu ouvrir les mains (gauche/droite) ?	Regarde-moi.
Peux-tu fermer les mains (gauche/droite) ?	Regarde ailleurs.
Mets tes mains (gauche/droite) en arrière.	Baisse les yeux (à gauche/droite).
Mains en l'air !	

Caractéristiques de Pepper	
Comment tu t'appelles ?	Est-ce que je peux te faire confiance ?
Quel est ton surnom ?	Es-tu humain ?
Présente-toi.	Es-tu vivant ?
Quelle taille fais-tu ?	Es-tu conscient ?
Combien pèses-tu ?	Est-ce que tu peux penser ?
Quel âge as-tu ?	Peux-tu ressentir des émotions ?
Quand est ton anniversaire ?	Comment détectes-tu les émotions ?
Es-tu un garçon ou une fille ?	Est-ce que tu manges ?
Tu es en quelle matière ?	Est-ce que tu peux parler ?
De quelle couleur es-tu ?	Est-ce que tu peux me voir ?
Pourquoi tu as une tablette ?	Est-ce que tu peux m'entendre ?
Es-tu gentil ?	Est-ce que tu peux me comprendre ?
Es-tu cool ?	Est-ce que tu peux me reconnaître ?
Es-tu intelligent ?	
A propos des robots	
Es-tu un robot ?	Qui est-ce qui t'a appelé comme ça ?
C'est quoi un robot ?	Qui t'a fabriqué ?
C'est quoi un robot humanoïde ?	Où as-tu été fabriqué ?
Est-ce que les robots vont remplacer les humains ?	As-tu de la famille ?
Connais-tu les lois de la robotique ?	As-tu des amis ?
Connais-tu Nao ?	Comment s'appelle ton ami ?
Connais-tu Roméo ?	Je peux être ton ami ?
Pourquoi tu t'appelles Pepper ?	

ANNEXE B

Publications de l'Humanoid Open Brest

L'événement a un site Internet [Zarrouki et al., 2018] regroupant la description de l'événement, les résultats des matchs de foot et une présentation des équipes. Un événement Facebook [Leboucher, 2018a] a été créé pour l'occasion. Les matchs ont été filmés par un robot cameraman [Pixio, 2018]. De nombreux articles ont été publiés : sur LinkedIn [Leboucher, 2018b], le Télégramme [Cessou, 2017][Cessou, 2018b] [Cessou, 2018a] et le Ouest-France [Floch, 2018]. France 3 a également consacré un reportage sur l'Humanoid Open Brest lors d'un journal régional.

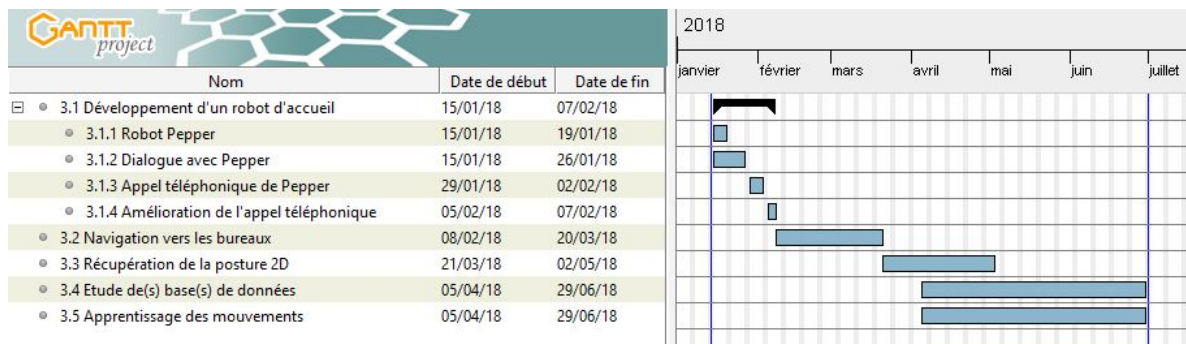


Annexe B : exemple d'un article du Télégramme

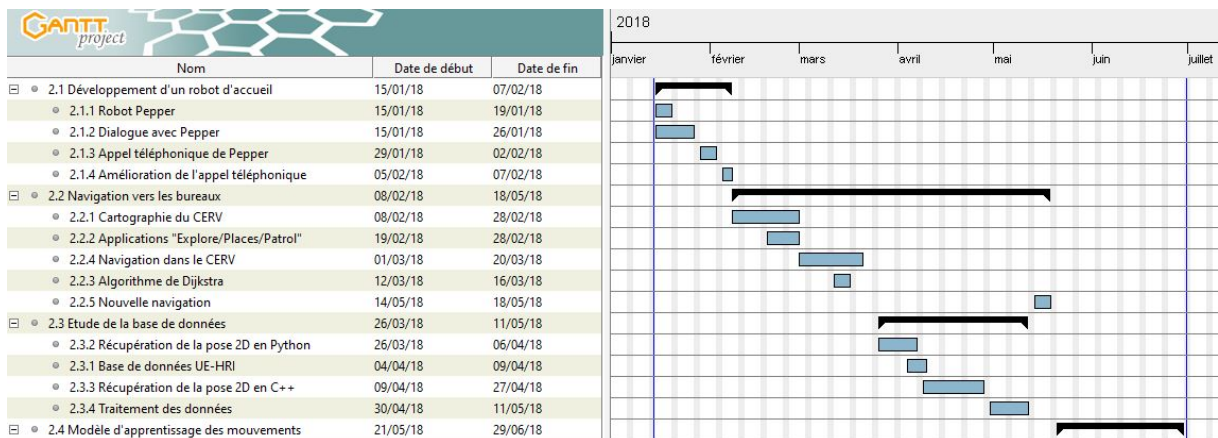
ANNEXE C

Diagrammes de Gantt prévisionnel et réel

Les deux diagrammes de Gantt ci-dessous représente le planning prévisionnel et le déroulement réel. Pour la partie robot d'accueil, les travaux devaient se finir vers la fin du mois d'avril. Dans les faits, la navigation a été reprise et s'est réellement terminée mi-mai. Le test des modèles, les installations et la création du modèle prennent du temps. Les réseaux de neurones sont complexe à comprendre et à bien utiliser donc le réseau est toujours en cours de test.



Annexe C : diagramme de Gantt prévisionnel



Annexe C : diagramme de Gantt réel

Bibliographie

- [Aldebaran, 2018] Aldebaran, S. R. (2018). Softbank Robotics documentation. <http://doc.aldebaran.com/2-5/index.html>, consulté le 21 mai 2018.
- [Alletto and Rigazio, 2017] Alletto, S. and Rigazio, L. (2017). Unsupervised motion flow estimation by generative adversarial networks.
- [Baltrušaitis et al., 2016] Baltrušaitis, T., Robinson, P., and Morency, L.-P. (2016). Openface : an open source facial behavior analysis toolkit. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1–10. IEEE.
- [Ben-Youssef et al., 2017] Ben-Youssef, A., Clavel, C., Essid, S., Bilac, M., Chamoux, M., and Lim, A. (2017). Ue-hri : a new dataset for the study of user engagement in spontaneous human-robot interactions. In *Proceedings of the 19th ACM International Conference on Multimodal Interaction*, pages 464–472. ACM.
- [Cao et al., 2017] Cao, Z., Simon, T., Wei, S.-E., and Sheikh, Y. (2017). Realtime multi-person 2d pose estimation using part affinity fields. In *CVPR*, volume 1, page 7.
- [Cessou, 2017] Cessou, M. (2017). Cerv-enib. des robots footballeurs. *Le Télégramme*. <http://www.letelegramme.fr/finistere/plouzane/cerv-enib-des-robots-footballeurs-11-12-2017-11774978.php>.
- [Cessou, 2018a] Cessou, M. (2018a). Humanoïd open brest. des robots jouent au foot à la mairie! *Le Télégramme*. <http://www.letelegramme.fr/finistere/brest/humanoid-open-brest-des-robots-jouent-au-foot-a-la-mairie-20-04-2018-11933156.php>.
- [Cessou, 2018b] Cessou, M. (2018b). Humanoïd open brest. une compétition de robots footballeurs à l’hôtel de ville. *Le Télégramme*. <http://www.letelegramme.fr/finistere/brest/humanoid-open-brest-une-competition-de-robots-footballeurs-a-l-hotel-de-ville-18-04-2018-11930570.php>.
- [Floch, 2018] Floch, A. (2018). Brest. maintenant, les robots se mettent au foot! *Ouest-France*. <https://www.ouest-france.fr/bretagne/brest-29200/brest-maintenant-les-robots-se-mettent-au-foot-5708247>.
- [Hanna and Stone, 2017] Hanna, J. P. and Stone, P. (2017). Grounded action transformation for robot learning in simulation. In *AAAI-17*, pages 3834–3840.
- [Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8) :1735–1780.
- [Huang and Mutlu, 2014] Huang, C.-M. and Mutlu, B. (2014). Learning-based modeling of multimodal behaviors for humanlike robots. In *Proceedings of the 2014 ACM/IEEE international conference on Human-robot interaction*, pages 57–64. ACM.

- [Huang and Khan, 2017] Huang, Y. and Khan, S. M. (2017). Dyadgan : Generating facial expressions in dyadic interactions. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 2259–2266. IEEE.
- [Jason et al., 2016] Jason, J. Y., Harley, A. W., and Derpanis, K. G. (2016). Back to basics : Unsupervised learning of optical flow via brightness constancy and motion smoothness. In *Computer Vision–ECCV 2016 Workshops*, pages 3–10. Springer.
- [Kober and Peters, 2010] Kober, J. and Peters, J. (2010). Imitation and reinforcement learning. *IEEE Robotics & Automation Magazine*, 17(2) :55–62.
- [Lasson et al., 2017] Lasson, F., Polceanu, M., Buche, C., and De Loor, P. (2017). Temporal deep belief network for online human motion recognition. *FLAIRS*.
- [Leboucher, 2018a] Leboucher, F. (2018a). Humanoïd Open Brest. Facebook event. <https://fr-fr.facebook.com/events/2041449262847192/>.
- [Leboucher, 2018b] Leboucher, F. (2018b). Robotique & intelligence artificielle : Humanoïd Open Brest du 18 au 20 avril salon Richelieu - Mairie de Brest - Accès libre de 13h30 à 17h30. Linkedin profile. <https://www.linkedin.com/pulse/robotique-intelligence-artificielle-humano%C3%AFd-open-brest-leboucher-1/?published=t>.
- [Martinez et al., 2017] Martinez, J., Hossain, R., Romero, J., and Little, J. J. (2017). A simple yet effective baseline for 3d human pose estimation. In *ICCV*.
- [Mehta et al., 2017] Mehta, D., Sridhar, S., Sotnychenko, O., Rhodin, H., Shafiei, M., Seidel, H.-P., Xu, W., Casas, D., and Theobalt, C. (2017). Vnect : Real-time 3d human pose estimation with a single rgb camera. *ACM Transactions on Graphics (TOG)*, 36(4) :44.
- [Mihoub et al., 2016] Mihoub, A., Bailly, G., Wolf, C., and Elisei, F. (2016). Graphical models for social behavior modeling in face-to face interaction. *Pattern Recognition Letters*, 74 :82–89.
- [Oh et al., 2015] Oh, J., Guo, X., Lee, H., Lewis, R. L., and Singh, S. (2015). Action-conditional video prediction using deep networks in atari games. In *Advances in Neural Information Processing Systems*, pages 2863–2871.
- [Olah, 2015] Olah, C. (2015). Understanding LSTM Networks. <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>, consulté le 07 juin 2018.
- [Paschalis and Lanius, 2018] Paschalis, P. and Lanius, C. (2018). Pyopenpose. <https://github.com/FORTH-ModelBasedTracker/PyOpenPose>.
- [Pixio, 2018] Pixio (2018). Un robot cameraman filme un match de foot de robots. Facebook post 19 avril 2018. <https://fr-fr.facebook.com/personalcameraman/posts/2133494416884282>.
- [PyTorch, 2018] PyTorch (2018). Site internet de PyTorch. <https://pytorch.org/>, consulté le 12 juin 2018.
- [Radford et al., 2015] Radford, A., Metz, L., and Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv :1511.06434*.

- [Souchet, 2017] Souchet, L. (2017). naoqi_navigation_samples. https://github.com/aldebaran/naoqi_navigation_samples.
- [Sukhbaatar et al., 2011] Sukhbaatar, S., Makino, T., Aihara, K., and Chikayama, T. (2011). Robust generation of dynamical patterns in human motion by a deep belief nets. In *Asian Conference on Machine Learning*, pages 231–246.
- [Twilio, 2018] Twilio (2018). Site internet de Twilio. <https://www.twilio.com/>, consulté le 21 mai 2018.
- [Wan et al., 2017] Wan, C., Probst, T., Van Gool, L., and Yao, A. (2017). Crossing nets : Combining gans and vaes with a shared latent space for hand pose estimation. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*
- [Yang et al., 2015] Yang, Y., Li, Y., Fermüller, C., and Aloimonos, Y. (2015). Robot learning manipulation action plans by " watching " unconstrained videos from the world wide web. In *AAAI*, pages 3686–3693.
- [Zarrouki et al., 2018] Zarrouki, C., Belejia, I., and Belouch, R. (2018). Site internet de l’Humanoid Open Brest. <https://www.enib.fr/robocup/>, consulté le 30 mai 2018.